2.2 Базовые типы

Целочисленные типы

int, int8, int16, int32, int64, uint, uint8, uint16, uint32, uint64

```
var a int = 42 // min = -2^63, max = 2^63 - 1
var b int8 = -128 // min = -2^7, max = 2^7 - 1
var c int16 = 32767 // min = -2^15, max = 2^15 - 1
var d int32 = -2147483648 // min = -2^31, max = 2^31 - 1
var e int64 = 9223372036854775807 // min = -2^63, max = 2^63 - 1

var f uint = 42 // max = 2^64 - 1
var g uint8 = 255 // max = 2^8 - 1
var h uint16 = 65535 // max = 2^16 - 1
var i uint32 = 4294967295 // max = 2^32 - 1
var j uint64 = 18446744073709551615 // max = 2^64 - 1
```

MERION

Использование различных систем счисления:

- **0b** или **0B**: двоичная система счисления (binary)
- **0o** или **0O**: восьмеричная система счисления (octal)
- **0x** или **0X**: шестнадцатеричная система счисления (hexadecimal)

```
package main
import "fmt" func
main() {
    binary := 0b101011 // Двоичное число
    octal := 0777      // 0o777 Восьмеричное число
    hexadecimal := 0x1A3F // Шестнадцатеричное число

    fmt.Printf("Двоичное: %b " , binary)
    fmt.Printf("Двоичное в десятичной: %d\n" , binary)

    fmt.Printf("Восьмеричное: %o " , octal)
    fmt.Printf("Восьмеричное в десятичной: %d\n" , octal)

    fmt.Printf("Шестнадцатеричное: %x " , hexadecimal)
    fmt.Printf("Шестнадцатеричное в десятичной: %d\n" ,
    hexadecimal)
}
```

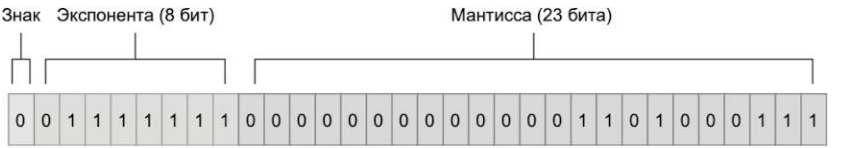
Использование подчеркиваний для читаемости:

```
package main func main() {

    bigNumber := 1_000_000 println(bigNumber)
}
```

Типы с плавающей точкой

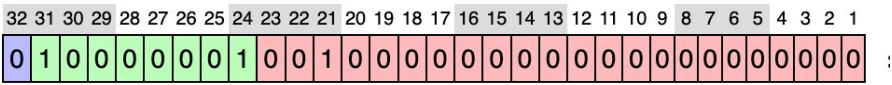
Стандарт IEEE 754:



`float32` , `float64`

```
var pi float32 = 3.14
var e float64 = 2.718281828459045
```

32-bit (float)



$1 \times 2^2 \times 1.125 = 4.5$

Три специальных вида чисел с плавающей точкой

- положительная бесконечность;
- отрицательная бесконечность;
- NaN (Not-a-Number — «не число») — результат неопределенной или непредставимой операции

Примеры создания и проверки:

```
inf :=
math.Inf(1)
 math.Inf(-1)

 nan :=
 math.Sqrt(-1.0)
 fmt.Println(nan)
```

Комплексные числа

`complex64` , `complex128`

```
var a complex64 = 1 + 2i
var b complex128 = 3 + 4i
```

MERION

Булевый тип

```
var a bool = true
var b bool = false
```

Строковый тип

```
var a string = "Hello, Go!"
var b string = `Multiline string`
```

Кодировка символов (charset)

Это просто набор символов. Кодировка ASCII содержит 128 (2^7) символов.

ASCII TABLE

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[END OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	|
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

MERION

Кодирование

Кодирование — это перевод списка символов в двоичный код.

UTF-8 — это стандарт кодирования, определяющий способ того, как возможно закодировать все символы в переменном количестве байтов (от 1 до 4 байт).

https://symbl.cc/en/unicode-table/

Типы для символов

`byte` (синоним `uint8`)

`rune` (синоним `int32`, представляет Unicode-символ)

```
var a byte = 'A' // ASCII символ
var b rune = '世' // Unicode символ
```

```
package main

import "unicode/utf8" import "fmt"

func main() {
    s := "世界" fmt.Println(len(s))
    fmt.Println(utf8.RuneCountInString (s))

    s = "Hello" fmt.Println(len(s))
    fmt.Println(utf8.RuneCountInString (s))
}
```

MERION

```
package main
import "fmt" func main() {

    s := "世界"
    for i := range s {
        fmt.Printf("position %d: %c\n", i, s[i])
    }
}
```

```
package main
import "fmt" func main()

{

    s := "世界"
    for index, runeValue := range s {
        fmt.Printf("%#U starts at byte %d\n" , runeValue, index)
    }
}
```

Тип any

Тип any — это **псевдоним (alias)** для interface{} .
Любой тип в Go автоматически его реализует.

```
package main

  func printValue(v any) {
     printf("Тип: %T, Значение: %v\n", v, v)
  }

  func main() {
     printValue(42)
     printValue("Hello")
     printValue(3.14)
  }
```

MERION